

PRISMtrace

by Block Convey

Getting Started Guide

Connect your AI agent, configure guardrails, and see every execution step — in under 15 minutes.

GETTING STARTED GUIDE | 2026

■ Block Convey

Connect your AI agent to PRISMtrace and get full visibility into every agent decision, guardrail block, and quality score — from day one.

WHAT PRISMTRACE GIVES YOU

OBSERVE Every agent interaction visible — quality scores, latency, cost, guardrail flags, token counts. <i>→ Enables per-agent SLA enforcement; feeds data warehouse via export destinations.</i>	GOVERN Prompt versioning creates a full audit trail — who changed instructions, when, and the quality impact. <i>→ Tracks prompt_id, author, timestamp, and quality delta per version.</i>
PROTECT PII/PHI detection and prompt injection blocking intercept threats before the LLM sees them. <i>→ Intercepts inputs before forwarding to the model — zero changes to LLM call logic.</i>	ALERT Real-time alerts fire the moment quality drops or guardrail block rates spike. <i>→ Email delivery via Resend, recipients configurable per rule; Slack and PagerDuty webhooks on the roadmap.</i>
TRACE Trajectory shows every step your agent took — data retrieval, guardrail check, LLM call — with individual latencies. <i>→ Auto-assembled from traces with a shared session_id; no separate endpoint required.</i>	REPORT Auto-generated compliance reports mapped to NIST AI RMF, SR 11-7, NAIC, NYDFS, HIPAA AI, CFPB, EU AI Act. <i>→ Export as PDF or deliver to auditor portal — no manual data assembly.</i>

1. GET YOUR API KEY (~2 MIN)

From the dashboard

- **API Keys** in the sidebar → Generate API Key → copy immediately. Shown once only.
- **Settings** → Project tab → copy your Project ID (UUID format).
- **Store both in a .env file.** Never put them in source code.

Quick validation (60 seconds)

Before writing SDK code, confirm credentials work:

```
export PRISMTRACE_API_KEY="pt-sk-..."
export PRISMTRACE_PROJECT_ID="uuid..."

curl -X POST https://prismtrace.blockconvey.com/api/traces \
-H "Content-Type: application/json" \
-d '{
  "api_key": "'$PRISMTRACE_API_KEY'",
  "project_id": "'$PRISMTRACE_PROJECT_ID'",
  "input_messages": [{"role": "user", "content": "Hello"}],
  "output_message": "Hi there!",
  "model": "gpt-4o",
  "latency_ms": 500
}'
```

Open Traces — your test trace appears within seconds.

Python — basic integration

■ Block Convey

Every PRISMtrace integration sends the same five identifiers from the start: `project_id`, `api_key`, `session_id`, `user_identifier`, and `step`. `session_id` ties traces in a single user turn into one trajectory. `user_identifier` populates the Users dashboard. `step / step_order` label what each trace represents.

```
pip install blockconvey-monitor>=0.1.3

import os, uuid, requests
from dotenv import load_dotenv
load_dotenv()

def send_trace(user_input, output, model, latency_ms, user_id):
    turn_id = str(uuid.uuid4()) # one UUID per user turn
    requests.post(
        'https://prismtrace.blockconvey.com/api/traces',
        json={
            'project_id': os.getenv('PRISMTRACE_PROJECT_ID'),
            'api_key': os.getenv('PRISMTRACE_API_KEY'),
            'session_id': turn_id,
            'user_identifier': user_id,
            'input_messages': [{'role': 'user', 'content': user_input}],
            'output_message': output,
            'model': model,
            'latency_ms': latency_ms,
            'metadata': {
                'agent_name': 'My Agent',
                'agent_id': 'my-agent-001',
                'step': 'llm_call',
                'step_order': 1,
            }
        },
        timeout=10,
    )

send_trace("Balance?", "$5,432.", "gpt-4o", 320, user_id="user_42")
```

✓ Top-level vs metadata

`session_id` and `user_identifier` are accepted at the top level OR inside metadata. Top level is preferred. `step` and `step_order` live in metadata.

Non-blocking (production)

Wrap `send_trace()` in a daemon thread to avoid blocking your agent:

```
import threading

def send_trace_async(user_input, output, model, latency_ms, user_id):
    threading.Thread(
        target=send_trace,
        args=(user_input, output, model, latency_ms, user_id),
        daemon=True,
    ).start()

# Returns immediately
send_trace_async("Balance?", "$5,432.", "gpt-4o", 320, user_id="user_42")
```

✓ Always fire-and-forget.

Never block the agent response path with a synchronous POST.

2. CONNECT YOUR AGENT — LANGGRAPH SDK

■ Block Convey

Three lines — no graph changes needed

```
from blockconvey.integrations.langgraph import BlockConveyCallbackHandler

handler = BlockConveyCallbackHandler(
    api_key=os.getenv('PRISMTRACE_API_KEY'),
    project_id=os.getenv('PRISMTRACE_PROJECT_ID'),
    agent_name='My Agent',
)

# Pass when invoking — graph unchanged
graph.invoke(state, config={'callbacks': [handler]})
```

✓ That is the complete LangGraph integration.

Every LLM call inside your graph now flows to PRISMtrace. Traces appear in the dashboard within seconds.

Supported integrations

- **LangChain** — callback handler, zero graph changes
- **LangGraph** — callback handler, zero graph changes
- **Anthropic SDK** — direct client wrapper
- **OpenAI SDK** — direct client wrapper
- **AWS Bedrock** — runtime wrapper
- **LiteLLM** — proxy wrapper
- **Node.js** — axios/fetch to REST endpoint
- **cURL** — REST endpoint directly

Required fields in every trace

- **project_id** — your project UUID
- **api_key** — your pt_... API key
- **input_messages** — array of {role, content} objects
- **output_message** — string response from your agent

Recommended on every trace

- **session_id** — UUID per user turn; ties traces into one trajectory
- **user_identifier** — your end-user ID; populates the Users dashboard
- **model** — model name for cost tracking
- **latency_ms** — integer milliseconds
- **token_count_input / token_count_output** — cost analytics
- **metadata.step / metadata.step_order** — labels each trace as a step in the trajectory
- **metadata.agent_name / metadata.agent_id** — displayed in Agent Fleet

3. CONFIGURE GUARDRAILS AND ALERTS (~5 MIN)

Built-in guardrail types

- **PII / PHI Detection** SSNs, credit cards, emails, phone numbers, health data
- **Prompt Injection** Adversarial prompts attempting to override agent instructions
- **Content Moderation** Harmful or inappropriate language
- **Off-Topic Detection** Queries outside your agent's domain (Knowledge Base-linked; contact Block Convey support to enable for your project)
- **Custom Regex** Your own patterns — account numbers, internal codes

■ Block Convey

- **Language Restriction** Enforce response language constraints

✓ Start here

Enable PII Detection → Block and Prompt Injection → Block immediately. 30 seconds.
Covers the most critical risks.

Recommended alert rules

Metric	Condition	Why
Compliance Score	Below 80	Catches degradation early
Latency	Above 5000 ms	Identifies slow responses
Guardrail Blocks	Above 10 / hr	Detects attack patterns

Quality scores — every trace

- **Customer Satisfaction (0–100)** How well the agent addressed the request
- **Response Quality (0–100)** Technical accuracy, completeness, clarity
- **Compliance Risk (0–100)** Lower = riskier. Flags missing disclaimers, PII leakage
- **Risk Level** Critical / High / Medium / Low composite

Prompt library

- Every system prompt edit creates a new version
- Side-by-side diff between any two versions
- One-click rollback to any prior version

Never put API keys or PII in prompts.

Use environment variables for secrets — not the Prompt Library.

4. AGENT TRAJECTORY — AUTO-ASSEMBLED FROM TRACES

What is trajectory?

Trajectory is PRISMtrace's step-by-step execution view. When your agent queries a database, checks guardrails, and calls an LLM, trajectory shows each step in sequence with its latency and output.

✓ Trajectory assembles automatically.

Send each step as its own trace with a shared session_id. PRISMtrace groups them by session_id, orders them by metadata.step_order, and labels them from metadata.step and the trace's model. No separate endpoint, no extra code beyond the basic example above.

What you get at each level

■ Block Convey

Level	What you send	What you see
Single trace	The basic example above (already includes session_id and step)	1-step trajectory: LLM Call with real latency
Multi-step	One trace per step, all sharing the same session_id, each with its own step / step_order	Full sequence with total latency, ordered by step_order
Custom labels	Set the trace's model field to a recognized value (see below)	Step type labels: Data Retrieval, Guardrail Check, LLM Call

Multi-step example

Each step gets its own trace. All share the same session_id. PRISMtrace assembles them into a trajectory in the background.

```
def handle_message(user_input, user_id):
    turn_id = str(uuid.uuid4()) # shared by every step in this turn

    # Step 1 – database query
    t0 = time.time()
    data = your_db.query(user_input)
    ms0 = int((time.time() - t0) * 1000)
    requests.post	TRACE_URL, json={
        'project_id': PROJECT_ID,
        'api_key': API_KEY,
        'session_id': turn_id,
        'user_identifier': user_id,
        'input_messages': [{'role': 'user', 'content': user_input}],
        'output_message': f'{len(data)} rows',
        'model': 'databricks-query',
        'latency_ms': ms0,
        'metadata': {'step': 'data_retrieval', 'step_order': 1},
    })

    # Step 2 – LLM call
    t1 = time.time()
    resp = your_llm.call(data, user_input)
    ms1 = int((time.time() - t1) * 1000)
    requests.post	TRACE_URL, json={
        'project_id': PROJECT_ID,
        'api_key': API_KEY,
        'session_id': turn_id, # same as step 1
        'user_identifier': user_id,
        'input_messages': [{'role': 'user', 'content': user_input}],
        'output_message': resp,
        'model': 'claude-sonnet-4-20250514',
        'latency_ms': ms1,
        'metadata': {'step': 'llm_call', 'step_order': 2},
    })
    return resp
```

Model values → trajectory labels

- **databricks-query** Data Retrieval — Databricks
- **snowflake-query** Data Retrieval — Snowflake

■ Block Convey

- **guardrail** Guardrail Check
- **claude-sonnet-4-...** LLM Call — Claude Sonnet
- **gpt-4o** LLM Call — GPT-4o

✓ Compliance evidence

When a guardrail blocks, the trajectory ends there — LLM Call never appears. Timestamp, rule name, and matched text are recorded automatically for auditors.

INTEGRATIONS | EXPORT DESTINATIONS

What export destinations do

Export Destinations send your PRISMtrace trace data to your data warehouse so your data team can run SQL on it and build dashboards in their existing tools.

i Not for observing your database

These send data OUT of PRISMtrace. Observing database calls inside your agent uses trajectory instrumentation (Section 4 above).

- **Databricks** Writes to a Delta Lake table in your workspace
- **Snowflake** Writes to PRISMTRACE.PUBLIC.TRACES via MERGE
- **Amazon S3** Coming soon — archival and pipeline integration

Agent fleet

Every agent auto-detected from traces — no manual registration. Quality score, guardrail rate, latency, and last active shown per agent. Add description, owner, and environment via the pencil icon on any card.

Example query after Databricks export

```
SELECT agent_name,
       COUNT(*) AS traces,
       ROUND(AVG(quality_score), 1) AS avg_quality,
       SUM(CASE WHEN guardrail_blocked THEN 1 ELSE 0 END) AS blocks
FROM   prismtrace_traces
WHERE  created_at >= DATE_SUB(NOW(), INTERVAL 7 DAY)
GROUP BY agent_name
ORDER BY blocks DESC;
```

SECURITY | API KEY RULES

- ✓ Store in .env or platform secret manager
- ✓ Add .env to .gitignore before first commit
- ✓ Rotate keys periodically
- ✓ Separate projects per environment
- ✗ Never hardcode keys in source code
- ✗ Never commit .env to Git

QUICK REFERENCE

■ Block Convey

Item	Value
Platform URL	https://prismtrace.blockconvey.com
API Endpoint	POST https://prismtrace.blockconvey.com/api/traces
SDK Install	<code>pip install blockconvey-monitor>=0.1.3</code>
API Key Location	API Keys page – shown once, copy immediately
Project ID Location	Settings → Project tab
Required Fields	<code>project_id</code> , <code>api_key</code> , <code>input_messages</code> , <code>output_message</code>
Session ID	<code>session_id</code> (top-level) or <code>metadata.session_id</code> – UUID per user turn; assembles traces into a trajectory
User Identifier	<code>user_identifier</code> (top-level) or <code>metadata.user_identifier</code> – populates the Users dashboard
Trajectory Fields	<code>metadata.step</code> + <code>metadata.step_order</code> – per-trace labels; ordering for the trajectory view
Trajectory Model Names	<code>databricks-query</code> , <code>snowflake-query</code> , <code>guardrail</code> , or any LLM model name
Support	arunprasad@blockconvey.com – 24 hr response on business days